eMerger

One command to update your whole system

Roberto Antoniello & Edoardo Ferrari

April 16, 2026

Abstract

eMerger is a cross-platform command line tool that refreshes, upgrades and cleans a computer with a single command: `up`. It runs on **Linux**, **macOS** and **Windows**, auto-detects every package manager present on the host and invokes them in the right order, returning one unified summary at the end. No daemon. No YAML. No embedded DSL. Just a small, auditable script that drives the tools you already have.

This document is written to be readable by anyone: newcomers will find an explanation of what a package manager is, what `up` actually does on your machine and how to try it safely; experienced users will find a complete reference for every flag, profile, hook, plugin and integration point, plus a cookbook of real-world recipes.

Contents

1 How to read this document **4**

2 Why eMerger? **5**

2.1 The problem 5

2.2 The solution 5

2.3 Design principles 5

2.4 What eMerger is *not* 6

3 Platforms at a glance 6

4 Requirements 6
4.1 Linux . . . 6
4.2 macOS . . . 7
4.3 Windows . . . 7

5 Installation 7
5.1 Linux and macOS . . . 7
5.2 Windows . . . 8
5.3 Manual install . . . 8
5.4 Uninstall . . . 8

6 Self-update 9

7 Quickstart 9
7.1 Step 1 - dry-run . . . 9
7.2 Step 2 - real run . . . 9
7.3 Step 3 - see what changed . . . 9
7.4 Step 4 - export a report . . . 9
7.5 Step 5 (optional) - weekly unattended update . . . 9

8 Your first real run 10

9 Execution flow 10

10 Command line reference 11
10.1 Full flag catalogue . . . 11
10.2 Combining flags . . . 12

11 Features in depth 12
11.1 Dry-run and verbose . . . 12
11.2 Quiet levels . . . 13
11.3 Security-only updates . . . 13
11.4 Dev toolchains . . . 13
11.5 Firmware (Linux) . . . 13
11.6 Parallel mode . . . 13
11.7 Snapshots and rollback (Linux) . . . 14
11.8 Mirrors refresh (Linux) . . . 14
11.9 Reboot handling . . . 14
11.10 Resume . . . 14
11.11 Download-only / offline . . . 15
11.12 Package diff and changelog . . . 15
11.13 Reports, history and errors . . . 15
11.14 Doctor . . . 15

12 Configuration 16
12.1 Config file (Unix) . . . 16
12.2 Config file (Windows) . . . 16
12.3 Every supported variable . . . 16
12.4 Profiles . . . 17

13 Hooks 18
13.1 Example: backup your dotfiles before every run . . . 18

13.2 Example: Slack notification after every run . . . 18
13.3 Example: copy the last run log to the clipboard (Windows) . . . 18
13.4 Example: auto self-update before upgrading . . . 18
13.5 Ignore list (Linux) . . . 18
13.6 Quiet hours . . . 19

14 Manager plugins **19**
14.1 Minimal plugin . . . 19
14.2 What you get for free . . . 19
14.3 Detection cache . . . 19

15 Integration **20**
15.1 JSON output . . . 20
15.2 Prometheus metrics . . . 20
15.3 Reboot exit code . . . 20
15.4 Download-only and offline . . . 21
15.5 Manager filtering . . . 21

16 Auto-update (unattended) **21**
16.1 Managing the schedule . . . 22

17 Cookbook **22**
17.1 Daily driver (Linux desktop) . . . 22
17.2 Unattended server . . . 22
17.3 Developer workstation . . . 22
17.4 Pre-demo machine . . . 23
17.5 Metered / mobile connection . . . 23
17.6 CI runner / container image builder . . . 23
17.7 Gated maintenance window . . . 23
17.8 Slack-notified runs . . . 23
17.9 macOS with Homebrew only . . . 23
17.10 Windows without Chocolatey . . . 23
17.11 Audit a single package . . . 24
17.12 Pinning a package (Linux) . . . 24

18 Security model **24**

19 Files and paths **24**

20 Supported package managers **25**

21 Exit codes **25**

22 Troubleshooting **26**

23 Frequently asked questions **27**

24 Glossary **27**

25 Development **28**
25.1 Repository layout . . . 28
25.2 Bash library modules (`src/lib/`) . . . 28
25.3 PowerShell library modules (`src/pslib/`) . . . 29
25.4 Running tests . . . 29

25.5 Adding a package manager 30
25.6 Contributing 30

1 How to read this document

The document is organised in layers:

- **Getting started** (sections 2 – 8): what eMerger does, why you might want it, and a guided first run that takes less than five minutes.
- **User manual** (sections 9 – 11): the execution flow, every command line flag with an example, and a feature-by-feature walkthrough.
- **Configuration** (sections 12 – 14): config file, profiles, hooks and user plugins, all with copy-pasteable snippets.
- **Integration** (sections 15 – 16): JSON, Prometheus, exit codes, CI, cron and systemd timers.
- **Cookbook** (section 17): ready-to-use recipes for laptops, servers, developer workstations, presentation machines, CI runners and metered connections.
- **Reference** (sections 19 – 24): file locations, supported managers, exit codes, troubleshooting, FAQ and a glossary of terms.
- **Development** (section 25): repo layout, module responsibilities, running tests, adding a package manager.

Throughout the document you will see three kinds of call-outs:

Tip - a useful shortcut or a better way of doing something.

Note - background information or a clarification that is worth remembering but not strictly required to move on.

Warning - something that can surprise you or cost you data if ignored. Read these carefully.

2 Why eMerger?

2.1 The problem

A modern desktop or laptop runs software that comes from many different sources at once:

- the operating system distributor (`apt`, `dnf`, `pacman`, `zypper`, `softwareupdate`, `winget` . . .);
- a cross-distro app store (`flatpak`, `snap`, `brew -cask`, `choco`, `scoop`);
- a user-level store (`brew`, `mas`);
- language-specific ecosystems (`npm`, `pip`, `cargo`, `gem`, `pnpm` . . .);
- vendor firmware (`fwupdmgr` on Linux, `softwareupdate` on macOS, Dell/Lenovo utilities on Windows).

Each uses a slightly different command, has its own cache, its own notion of "security update" and its own definition of "clean". Keeping all of them up to date by hand is either tedious or error-prone, and writing a personal wrapper that does the right thing on three operating systems is a weekend project most people never finish.

2.2 The solution

eMerger is that wrapper, generalised. Typing `up` on any supported platform will:

1. detect every package manager installed on the host;
2. ask for `sudo` (or trigger a UAC elevation on Windows) only if one of the detected managers actually needs it;
3. take an optional filesystem snapshot or a package list snapshot so you can audit the run afterwards;
4. run refresh / upgrade / clean, in the right order, serially or in parallel where safe;
5. clean user caches and the trash, optionally;
6. print a boxed summary with per-manager result, disk freed, run duration and a reboot advisory;
7. export the same summary as JSON, Markdown or Prometheus textfile if you ask.

It does this with no daemon, no YAML, no Python runtime and no Ruby runtime: just a Bash script on Unix and a PowerShell script on Windows.

2.3 Design principles

- **One command.** The user types `up`; everything else is automatic detection, ordering and reporting.
- **No runtime bloat.** Configuration is a plain shell or PowerShell file that is sourced at startup.
- **Use what is there.** eMerger never installs a package manager on your behalf; it detects the ones already installed and delegates work to them.
- **Safe by default.** Dry-run, retries, filesystem snapshots, resume after interruption and a pre-flight health check are all first-class options.

- **Cross-platform parity.** The core flow (detect, upgrade, clean, summarise) is identical on Linux, macOS and Windows. Platform-specific features are gated and labelled.

2.4 What eMerger is *not*

- It is **not** a package manager itself. It does not download or install software directly: it asks the native tools to do it.
- It is **not** a configuration management system. There is no desired-state manifest, no idempotent resource graph. If you want that, use Ansible, Salt or NixOS.
- It is **not** a scheduler. It can install a weekly systemd timer, cron job or Windows scheduled task, but it does not run as a service.

The project name is a wink at Gentoo's `emerge`: the tool started under the name *Updater* and was later renamed to *eMerger* to reflect its goal of merging the quirks of every supported package manager behind a uniform interface.

3 Platforms at a glance

	Linux	**macOS**	**Windows**
Entry point	`src/emerger.sh`	`src/emerger.sh`	`src/emerger.ps1`
Shell	bash 3.2+	bash 3.2+ (system)	PowerShell 5.1+
Setup	`./setup.sh`	`./setup.sh`	`.\setup.ps1`
Uninstall	`./uninstall.sh`	`./uninstall.sh`	`.\uninstall.ps1`
Auto-update	systemd timer / cron	cron	Task Scheduler
Elevation	`sudo`	`sudo`	UAC relaunch
TUI menu	yes (`-i`)	yes (`-i`)	no
Parallel mode	yes	yes	no (always serial)
Snapshots	snapper/timeshift/btrfs	no	no (System Restore manual)
Config dir	`~/.config/emerger/`	`~/.config/emerger/`	`%APPDATA%\emerger\`
State dir	`~/.local/state/emerger/`	`~/.local/state/emerger/`	`%LOCALAPPDATA%\emerger\state\`

The core flow is identical across platforms. Features that depend on operating system primitives (firmware updates, filesystem snapshots, TUI) are gated to the platforms that support them and labelled as such throughout this document.

4 Requirements

eMerger never installs dependencies for you. The list below is what "works out of the box" vs. what unlocks optional features.

4.1 Linux

Required: `bash` 3.2+, `coreutils`, `git`, `sudo`.

Optional:

- `gum` or `whiptail` for a nicer interactive menu.
- `notify-send` for desktop notifications.
- `curl` for the weather widget (`-w`) and `-self-update` checks.

- `flock` for the global lock (pre-installed on every mainstream distribution).
- `reflector` (Arch) or `netselect-apt` (Debian / Ubuntu) for `-refresh-mirrors`.
- `snapper`, `timeshift` or `btrfs-progs` for `-snapshot` and `-rollback`.
- `fwupdmgr` for `-firmware`.

4.2 macOS

Required: the system `bash` at `/bin/bash` (3.2+ is enough; no Homebrew Bash needed), and Xcode Command Line Tools for `git`.

Optional:

- Homebrew (`brew`), which in turn enables `brew -cask`.
- `mas` (`brew install mas`) for Mac App Store upgrades.

4.3 Windows

Required: PowerShell 5.1 (built-in on Windows 10 and later) or 7+, Git for Windows (needed by `up -self-update`).

Optional:

- `winget`, `scoop`, `choco`: any combination is auto-detected.
- `PSWindowsUpdate` (`Install-Module PSWindowsUpdate -Scope CurrentUser`) for Windows Update integration.
- `BurntToast` (`Install-Module BurntToast -Scope CurrentUser`) for native toast notifications.

> eMerger never installs these for you - it only uses what is already installed. This is deliberate: `up` must never put software on your machine that you didn't ask for.

5 Installation

5.1 Linux and macOS

```
git clone https://github.com/MasterCruelty/eMerger
cd eMerger
./setup.sh
# open a new shell, or: source ~/.bashrc
up --help
```

`setup.sh` performs the following actions, in order:

1. Adds `alias up='bash /path/to/eMerger/src/emerger.sh'` to the appropriate shell rc file (`.bashrc`, `.zshrc`, `config.fish`). If you use multiple shells it writes the alias in all of them.
2. Marks `src/emerger.sh` as executable.
3. Installs shell completions for `bash`, `zsh` and `fish`:
   - `~/.local/share/bash-completion/completions/up`

- ~/.zsh/completions/_up
- ~/.config/fish/completions/up.fish

4. Scaffolds ~/.config/emerger/ with the default config.sh, ignore.list, hooks/ and profiles.d/ layout.

On macOS the alias is written to ~/.zshrc first, since zsh is the default interactive shell from Catalina onwards. When Homebrew is present, completions are installed under the Homebrew prefix so that they are loaded by the brew-managed shell environment.

5.2 Windows

```
git clone https://github.com/MasterCruelty/eMerger
cd eMerger
.\setup.ps1
# open a new PowerShell window, or: . $PROFILE
up --help
```

setup.ps1 does *not* require administrator privileges:

1. Lifts ExecutionPolicy for CurrentUser to RemoteSigned when it was Restricted or Undefined.
2. Defines function up { & "...\emerger.ps1" @args } inside $PROFILE.CurrentUserAllHosts, so up works from every PowerShell host (cmd-hosted PS, ISE, Windows Terminal, VS Code integrated terminal).
3. Scaffolds %APPDATA%\emerger\ with config.ps1, hooks\pre.d\, hooks\post.d\, profiles.d\.

When a package manager run later needs elevation, eMerger relaunches itself under UAC. The relaunch is logged, so you always know when it happened.

5.3 Manual install

When a packaged setup is undesirable (read-only home, corporate lockdown, portable installation), pointing an alias at the entry point is enough:

```
# bash, zsh, fish - Linux or macOS
alias up='bash /absolute/path/to/eMerger/src/emerger.sh'
```

```
# PowerShell - Windows
function up { & "C:\path\to\eMerger\src\emerger.ps1" @args }
```

5.4 Uninstall

```
./uninstall.sh      # Linux / macOS
.\uninstall.ps1     # Windows
```

The uninstaller removes the shell alias, the cron job or systemd timer, and the scheduled task. Configuration and state directories are **kept** so that logs, profiles and hooks survive a reinstall; remove them manually to start from a clean slate:

```
rm -rf ~/.config/emerger ~/.local/state/emerger ~/.cache/emerger
```

The repository itself is not removed: rm -rf the directory when you are done.

6 Self-update

Three equivalent forms are accepted:

```
up -up                # short flag
up --self-update      # long flag
./update.sh           # out-of-band script (.\update.ps1 on Windows)
```

Internally a `git pull -ff-only` is performed inside the clone and the new commit range is printed. Non fast-forward updates are refused so that local changes never silently disappear. If you want the self-update to also run every time you update your system, add it to a `post.d` hook (see section 13).

7 Quickstart

Five minutes. No commitment. Nothing gets upgraded until you say so.

7.1 Step 1 - dry-run

```
up -n
```

This is the safe preview. It shows which managers were detected and what commands *would* be invoked. No `sudo`, no downloads, no changes on disk.

7.2 Step 2 - real run

```
up
```

Refresh, upgrade, clean, one manager at a time. You will be asked for `sudo` once (Unix) or get a UAC prompt (Windows) only if a detected manager needs it.

7.3 Step 3 - see what changed

```
up --changed
```

Prints the package diff for the last run with + for added, − for removed and ~ for upgraded.

7.4 Step 4 - export a report

```
up --report last-run.md
```

Produces a Markdown file with the JSON summary, per-manager results, reboot advisory and the full package diff as a table.

7.5 Step 5 (optional) - weekly unattended update

```
up -au
```

Installs a systemd user timer on Linux, a cron job on macOS or a scheduled task on Windows. The trigger fires every Sunday at 10:00 (local time) with ±1h randomised delay, and always uses `-y -q -nl -ni`.

8 Your first real run

The very first time you run `up`, you should expect something like this on a Debian/Ubuntu desktop with Flatpak and a couple of dev toolchains:

```
$ up
 _____    __  __
|  ___|  |  \/  |  eMerger v2.0.0
| |__    | |\/| |  one command for the whole system
|____|   |_|  |_|
Ubuntu 24.04  o  x86_64  o  2026-04-16 10:12
eMerger v2.0.0  o  github.com/MasterCruelty/eMerger
[1/3] apt        OK     (38 upgraded, 0 removed, 0 new)
[2/3] flatpak    OK     (5 refreshed)
[3/3] fwupd      SKIP   (--firmware not set)
+----------------------------------------------+
|  eMerger - summary                           |
|  duration  42s                               |
|  freed     15 MiB                            |
|  reboot    not required                      |
|  errors    0                                 |
+----------------------------------------------+
```

If the colours or the box look wrong, your terminal may not support Unicode glyphs. Run `up -no-emoji` once, or add `ARG_NO_EMOJI=1` to your `config.sh`.

Do not run `up` with `sudo` at the top level. eMerger will ask for `sudo` itself, only for the managers that need it, and will run user-level managers (`brew`, `flatpak` user remotes, `npm` user globals, `pip -user` ...) *as you*, not as root. Calling `sudo up` will install user-level packages into root's home.

9 Execution flow

A full run of `up` performs, in order:

1. Load `config.sh` (or `config.ps1`) if present.
2. If `--profile NAME` is on the command line, pre-load `share/profiles/NAME.sh` (or the user override from `~/.config/emerger/profiles.d/`).
3. Parse CLI flags. CLI always wins over config and profile defaults.
4. If `QUIET_HOURS` is set and we are inside the window *and* `-y` is set, exit immediately (this shields scheduled runs from night-time activity without blocking interactive ones).
5. Acquire the global exclusive lock (`/tmp/emerger.lock` via `flock` on Unix). If another `up` is already running, abort with exit code 1.
6. Print the logo, OS info line and the current timestamp.
7. Warn on low battery (<20% on a laptop) and on low free disk space (default: less than 1024 MiB on `/` or `C:`).
8. Cache `sudo` credentials on Unix - or relaunch elevated on Windows - only when a detected manager actually needs it.
9. Snapshot the installed-packages list into `pkgs.before` for the post-run diff.

10. Execute `pre.d` hooks in alphabetical order.
11. For each detected manager, run *refresh*, *upgrade*, *clean*; serial for system managers, concurrent for safe user-space ones when `-parallel` is active.
12. Optionally clear user cache and trash (skippable with `-nc` and `-nt`).
13. Execute `post.d` hooks.
14. Snapshot the installed-packages list into `pkgs.after` and compute `pkgs.diff`.
15. Render the boxed summary, the reboot advisory and an optional desktop notification.
16. Exit 0 on success, 3 if any manager failed, 4 if a reboot is required and `-reboot-exit` was passed.

10 Command line reference

The authoritative reference is `up --help`; the table below summarises every flag and is annotated with the most common uses.

10.1 Full flag catalogue

Flag	Meaning	Platforms
`-h`, `-help`	Print help text and exit	all
`-V`, `-version`	Print `eMerger X.Y.Z`	all
`-n`, `-dry-run`	Preview without running	all
`-v`, `-verbose`	Stream manager output live	all
`-q` / `-qq` / `-qqq`	Progressive quiet levels	all
`-y`, `-yes`	Assume yes on prompts	all
`-i`, `-interactive`	TUI menu (gum / whiptail)	Unix
`-w`, `-weather`	Show weather line via wttr.in	all
`-no-emoji`	Force ASCII glyphs only	all
`-nl`, `-no-logo`	Hide logo	all
`-ni`, `-no-info`	Hide system info line	all
`-nc`, `-no-cache`	Skip user cache cleaning	all
`-nt`, `-no-trash`	Skip trash cleaning	all
`-security`	Security-only updates	supported managers
`-firmware`	Include `fwupdmgr`	Linux
`-no-firmware`	Force-skip firmware	Linux
`-dev`	Include dev toolchains	all
`-parallel`	Run user-space in parallel	Unix
`-profile NAME`	Load a profile	all
`-list-profiles`	List available profiles	all
`-only LIST`	Only these managers	all
`-except LIST`	Skip these managers	all
`-snapshot`	snapper / timeshift / btrfs	Linux
`-rollback`	Revert last eMerger snapshot	Linux
`-refresh-mirrors`	Re-rank distro mirrors	Linux
`-resume`	Skip completed managers	Unix
`-reboot`	Reboot if required	all
`-reboot-exit`	Exit 4 if reboot required	all

Flag	Meaning	Platforms
-download-only, -offline	Prefetch, don't install	Unix
-changed	Show package diff	Unix
-changelog PKG	Upstream changelog for PKG	Unix
-report [FILE]	Export Markdown report	Unix
-history	Recent runs	all
-err, -errors	Tail of logged errors	all
-doctor	Health check	all
-json	Machine-readable summary	all
-metrics FILE	Prometheus textfile export	all
-up, -self-update	Self-update eMerger	all
-au, -auto-update	Install weekly auto-update	all
-rc, -rebuild-cache	Clear detection cache	all

10.2 Combining flags

Flags are independent tokens and can be combined freely, in any order. Single-letter short flags in the set {h V n v q y i w} bundle (up -nv equals up -n -v; up -ynv equals up -y -n -v). Compound short flags (-nl, -ni, -qq, -up, -err . . .) and long flags are never bundled.

```
up -n -v                        # dry-run with live stream
up -y -q --security             # unattended security-only, minimal output
up --dev --parallel -v          # dev toolchains + user-space concurrency
up --snapshot --reboot -y       # snapshot first, reboot at the end if needed
up --profile server --resume    # resume an interrupted headless run
up -n --dev --firmware          # preview a full run, no side effects
up -qq -y -nl -ni --security    # exactly what the scheduled timer runs
up --refresh-mirrors -y -v      # re-rank mirrors then upgrade, live
up --changed --report out.md    # show diff and export in one shot
```

Flags that take a value (-profile NAME, -changelog PKG, -report FILE, -metrics FILE, -only LIST, -except LIST) must keep their argument adjacent; every other flag is position-free. CLI flags always win over config file and profile defaults, so you can override a profile on the fly:

```
up --profile work --dev         # work profile, but force --dev this time
```

11 Features in depth

11.1 Dry-run and verbose

-n prints every command that *would* be executed without actually running it or asking for elevation, which makes it safe to run in a terminal that does not own sudo credentials. Example output:

```
$ up -n
 [dry] sudo apt update
 [dry] sudo apt upgrade -y
 [dry] sudo apt autoremove -y
 [dry] sudo apt clean
 [dry] flatpak update -y
 [dry] flatpak uninstall --unused -y
```

-v streams the output of each package manager live instead of buffering it into the summary. The two combine naturally: up -n -v.

11.2 Quiet levels

- **default** - full UI with logo, info line, step headers, spinner, summary box.
- -q - hides muted and informational lines but keeps the per-manager result and the summary.
- -qq - only step titles and the one-line summary; suitable for cron/systemd logs.
- -qqq - no output at all; the exit code is the only signal. Suitable for embedding in shell pipelines.

11.3 Security-only updates

When -security is passed, each supporting manager is asked to limit the upgrade set to security advisories:

- **apt** via unattended-upgrade -security (falls back to apt upgrade for packages that are not in the unattended security tree).
- **dnf** with upgrade -security.
- **zypper** with patch -category security.
- **softwareupdate** (macOS) with -recommended.
- **PSWindowsUpdate** (Windows) with severity filtering.

Managers that do not distinguish between security and non-security updates simply ignore the flag and skip the run.

11.4 Dev toolchains

-dev enables a separate track that runs:

Tool	Command
rustup	rustup self update && rustup update stable
cargo	cargo install-update -a
npm	npm update -g
pnpm	pnpm -g update
pip	pip list -outdated, then user-site upgrade
gem	gem update

These run under the invoking user, never under sudo. If a toolchain isn't installed it is silently skipped.

11.5 Firmware (Linux)

-firmware runs fwupdmgr refresh && fwupdmgr update -y -no-reboot-check. If the LVFS (Linux Vendor Firmware Service) is not reachable the step is skipped with a warning. On macOS firmware is handled by softwareupdate; on Windows it is delegated to vendor tooling (Dell Command Update, Lenovo Vantage . . .) and is out of scope.

11.6 Parallel mode

-parallel runs user-space managers that do not touch the root filesystem concurrently: flatpak, snap, brew, mas, and the dev toolchains. System managers (apt, dnf, pacman, zypper . . .) stay

serial to avoid lock contention on `/var/lib/dpkg`, `/var/cache/pacman` and similar. The Windows side is currently always serial.

11.7 Snapshots and rollback (Linux)

`-snapshot` tries, in order:

1. `snapper` - creates a pre-update snapshot tagged `eMerger pre-update` on the root subvolume.
2. `timeshift` - runs `timeshift -create -tags D`.
3. raw `btrfs subvolume snapshot` - only if neither of the above is installed; the snapshot is placed under `/.snapshots/emerger/`.

`-rollback` reverts to the most recent eMerger-created snapshot:

- **snapper**: native `snapper -c root rollback <num>`. A reboot is required afterwards (snapper semantics, not ours).
- **timeshift**: hands off to `timeshift -restore` *interactively* - silent rollback is too destructive to automate.
- **raw btrfs**: refuses to swap subvolumes automatically, prints the path of the latest snapshot so you can do it manually.

Combine with a guard to get a safe update cycle:

```
up --snapshot -y || up --rollback
```

11.8 Mirrors refresh (Linux)

`-refresh-mirrors` re-ranks the repository mirrors before the upgrade:

- **Arch**: `reflector -latest 20 -sort rate`.
- **Debian/Ubuntu**: `netselect-apt` (requires `netselect-apt` installed).
- **Fedora**: no-op; the `fastestmirror` DNF plugin already ranks mirrors at download time.

11.9 Reboot handling

On Unix, eMerger inspects `/var/run/reboot-required` and `needs-restarting -r` (if available).
On Windows it reads the CBS `RebootPending`, `WindowsUpdate\RebootRequired` and `PendingFileRenameOperati` registry entries.

- The summary always prints a reboot advisory.
- `-reboot` actually reboots the machine (requires elevation).
- `-reboot-exit` exits with code **4** so that an orchestrator can decide what to do next.

11.10 Resume

When a run is interrupted (Ctrl-C, lost network, disk full, laptop suspend), a cursor is written to `~/.local/state/emerger/resume` after every successful manager. A subsequent `up -resume` skips every manager that completed successfully, so repeated failures do not force a full retry. Unix only.

11.11 Download-only / offline

`-download-only` (alias `-offline`) refreshes the package indexes and *downloads* the pending upgrade set but does not install it. Supported natively on `pacman` (`-Syuw`), `apt` (`-download-only`), `dnf` (`-downloadonly`) and `zypper` (`update -download-only`). Typical use cases:

- laptop on a slow or metered connection: prefetch while on good Wi-Fi, install later at the office;
- servers in a maintenance window: prestage the set, flip to install-only when the change ticket opens;
- pre-flight for `-snapshot`: verify the whole update set is downloaded *before* taking a snapshot.

11.12 Package diff and changelog

Each run records the installed package set before and after. `up -changed` prints the diff with `+`, `-` and `~` markers:

```
$ up --changed
 ~ firefox          123.0.1   ->  124.0
 ~ libc6            2.39-0    ->  2.39-1
 + flatpak-xdg-utils 1.0.6
 - old-package      2.1
```

`up -changelog PKG` dispatches to `apt changelog`, `dnf changelog` (or `dnf updateinfo` if available), `pacman -Qi` or `brew log` depending on the host.

11.13 Reports, history and errors

- `-report FILE` exports a Markdown report of the last run. Without a filename it writes to `$PWD/emerger-report.md`.
- `-history` prints the last 10 runs as a table.
- `-err` / `-errors` tails `ERROR` lines from `~/.local/state/emerger/emerger.log`.

11.14 Doctor

`-doctor` audits the environment and prints a traffic-light report:

- shell / PowerShell version;
- `sudo` cache / admin status;
- disk space on the state partition;
- network reachability (IPv4 and IPv6);
- state directory writability;
- per-manager native health: `dpkg -audit`, `pacman -Dk`, `brew doctor`, `pip check`;
- pending reboot flag;
- (Windows) the current ExecutionPolicy and elevation status.

A non-zero exit code is returned when any issue is detected. Always run `up -doctor` before opening a bug report.

12 Configuration

12.1 Config file (Unix)

`~/.config/emerger/config.sh` is a plain shell file sourced before argument parsing. Every flag has a corresponding `ARG_*` variable:

```
# ~/.config/emerger/config.sh

# Defaults
ARG_DEV=1                   # always include dev toolchains
ARG_WEATHER=1               # show weather widget via wttr.in
ARG_PARALLEL=1              # run user-space managers in parallel
ARG_NO_TRASH=1              # never empty the trash automatically

# Thresholds
DISK_MIN_FREE_MB=2048       # require at least 2 GiB free on /
BATTERY_MIN_PCT=25          # warn below 25%
RETRY_MAX=3                 # transient-error retry count
RETRY_DELAY=5               # seconds between retries

# Scheduling
QUIET_HOURS="23:00-07:00"   # scheduled runs exit immediately inside this window

# Caching
EMERGER_CACHE_TTL=3600      # detection cache: 1h (0 = disabled, default 86400)
```

12.2 Config file (Windows)

`%APPDATA%\emerger\config.ps1` is dot-sourced before argument parsing:

```
# %APPDATA%\emerger\config.ps1

$script:ArgsGlobal.Dev      = $true
$script:ArgsGlobal.Security = $true
$script:ArgsGlobal.NoTrash  = $true

# Thresholds (same semantics as Unix)
$script:DISK_MIN_FREE_MB = 4096
$script:RETRY_MAX = 3
```

12.3 Every supported variable

Variable	Default	Meaning
`ARG_DEV`	0	include dev toolchains by default
`ARG_FIRMWARE`	0	include `fwupdmgr` by default
`ARG_NO_FIRMWARE`	0	force-skip firmware
`ARG_SECURITY`	0	security-only by default
`ARG_YES`	0	assume yes by default
`ARG_PARALLEL`	0	parallel user-space by default
`ARG_WEATHER`	0	show weather widget
`ARG_NO_EMOJI`	0	force ASCII glyphs
`ARG_NO_CACHE`	0	skip user cache cleaning
`ARG_NO_TRASH`	0	skip trash cleaning

Variable	Default	Meaning
ARG_NO_LOGO	0	hide logo
ARG_NO_INFO	0	hide system info line
QUIET_LEVEL	0	0=full UI, 1=-q, 2=-qq, 3=-qqq
DISK_MIN_FREE_MB	1024	abort / warn below this many MiB
BATTERY_MIN_PCT	20	warn below this battery percent
RETRY_MAX	2	transient-error retries
RETRY_DELAY	3	seconds between retries
QUIET_HOURS	(unset)	"HH:MM-HH:MM" - skip scheduled runs inside window
EMERGER_CACHE_TTL	86400	detection-cache TTL in seconds; 0 disables

The CLI always wins. ARG_SECURITY=1 in config.sh plus up -dev on the CLI will run security updates *and* dev toolchains; passing -no-firmware will override ARG_FIRMWARE=1.

12.4 Profiles

Profiles are named configuration snippets loaded through -profile NAME. They are ordinary config-file fragments scoped to a name.

The shipped set lives under share/profiles/ and includes:

Profile	Meant for
work	laptop at the office - security-only, unattended, no cache or trash wiping
home	desktop at home - everything, including dev toolchains, firmware and parallel mode
server	headless - quiet (-qq), security only, no prompts, no UI
safe	before a demo or presentation - security only, no heavy downloads

User profiles go in ~/.config/emerger/profiles.d/ (Unix) or %APPDATA%\emerger\profiles.d\ (Windows). They shadow the shipped defaults: a user work.sh wins over share/profiles/work.sh.

Full content of the shipped work profile:

```
# description: laptop at work - security only, no firmware, never reboot
ARG_SECURITY=1
ARG_YES=1
ARG_NO_FIRMWARE=1
ARG_NO_CACHE=1
ARG_NO_TRASH=1
```

Writing a custom profile is just a shell fragment:

```
# ~/.config/emerger/profiles.d/train.sh
# description: on a train, prefetch only, keep the fans quiet
ARG_DOWNLOAD_ONLY=1
ARG_YES=1
ARG_QUIET=1
QUIET_LEVEL=2
ARG_NO_TRASH=1
ARG_NO_CACHE=1
```

Then:

```
up --profile train
```

List what is available:

```
up --list-profiles
```

13 Hooks

Executable scripts in `hooks/pre.d/` and `hooks/post.d/` run before and after the update flow, in alphabetical order. A failing hook emits a warning but does not abort the run, so hooks are safe by default. Unix hooks are `*.sh` under bash; Windows hooks are `*.ps1` dot-sourced under PowerShell.

13.1 Example: backup your dotfiles before every run

```
# ~/.config/emerger/hooks/pre.d/10-backup-dotfiles.sh
#!/usr/bin/env bash
set -e
rsync -a --delete ~/.config/ ~/backups/dotfiles/
```

13.2 Example: Slack notification after every run

```
# ~/.config/emerger/hooks/post.d/99-slack-notify.sh
#!/usr/bin/env bash
state=~/.local/state/emerger
tail -1 "$state/history.jsonl" | \
  curl -sS -X POST -H 'Content-Type: application/json' \
    --data-binary @- "$SLACK_WEBHOOK_URL" >/dev/null
```

13.3 Example: copy the last run log to the clipboard (Windows)

```
# %APPDATA%\emerger\hooks\post.d\10-log-to-clipboard.ps1
$log = Join-Path $env:LOCALAPPDATA 'emerger\state\emerger.log'
Get-Content $log -Tail 40 | Set-Clipboard
```

13.4 Example: auto self-update before upgrading

```
# ~/.config/emerger/hooks/pre.d/00-self-update.sh
#!/usr/bin/env bash
( cd "$(dirname "$(readlink -f "$(command -v up | awk '{print $NF}')")")/.." && git
    pull --ff-only ) || true
```

> Hooks run with the privileges that invoked `up`. A `pre.d` hook that writes outside of `$HOME` may silently fail until `sudo` has been cached later in the run. If you need root-owned side effects, write them in `post.d` and guard with `sudo -n` or a `sudoers` entry.

13.5 Ignore list (Linux)

`~/.config/emerger/ignore.list` accepts one package per line, with # comments. Behaviour per manager:

Manager	Behaviour
pacman	Honoured natively via -ignore= on the command line.
apt	Advisory: you must still apt-mark hold PKG.
dnf	Advisory: you must dnf versionlock add PKG.
zypper	Advisory: you must zypper al PKG.

13.6 Quiet hours

QUIET_HOURS="HH:MM-HH:MM" in config.sh makes scheduled runs that start inside the window exit immediately when -y is set (i.e. the ones coming from the timer). Interactive invocations are never blocked. The window may cross midnight: "23:00-07:00" is valid.

14 Manager plugins

Dropping a bash script in ~/.config/emerger/managers.d/<slug>.sh registers a custom manager without touching the repository.

14.1 Minimal plugin

```
# ~/.config/emerger/managers.d/mytool.sh
PM_PLUGIN_SLUG=mytool

pm_mytool_detect()     { command -v mytool >/dev/null 2>&1; }
pm_mytool_needs_sudo() { return 1; }     # optional, default: no sudo
pm_mytool_parallel()   { return 0; }     # optional, default: serial
pm_mytool_dev()        { return 1; }     # optional, default: not --dev gated
pm_mytool_icon()       { printf '\xf0\x9f\x94\x8c'; }   # optional

pm_mytool_run() {
    run_cmd "mytool refresh"  mytool refresh  || return 1
    run_cmd "mytool upgrade"  mytool upgrade -y || return 1
    return 0
}
```

14.2 What you get for free

Plugins are registered at the same level as native managers: they honour -only, -except, -parallel, -dev, the detection cache, the hooks and the summary. Because pm_<slug>_run calls run_cmd, your plugin automatically gets:

- -dry-run support;
- retry on transient failures (RETRY_MAX, RETRY_DELAY);
- logging to ~/.local/state/emerger/emerger.log;
- live-log streaming under -v.

A copy-pasteable template lives at share/plugins/example.sh.

14.3 Detection cache

The detection cache is keyed by manager slug and lives at ~/.cache/emerger/detected. TTL defaults to 1 day; override via EMERGER_CACHE_TTL=<seconds> in config.sh (0 disables caching entirely). After installing or removing a package manager run up -rc to clear the cache.

Plugins are Linux and macOS only. Windows plugins are not yet supported.

15 Integration

15.1 JSON output

`up -json` emits a single-line JSON object on stdout. The logo, info line and summary box are all suppressed, so the output is safe to pipe into `jq` or consume from a CI job:

```
{"ts":"2026-04-14T07:24:31Z","duration":42,"freed_kb":15360,
 "errors":0,"reboot":0,
 "managers":[{"name":"apt","result":"ok"},{"name":"flatpak","result":"ok"}]}
```

Every run also appends one such line to `~/.local/state/emerger/history.jsonl` (one JSON per line, a.k.a. JSONL). Fields:

Field	Meaning
`ts`	ISO 8601 UTC timestamp of the run start
`duration`	total wall-clock seconds
`freed_kb`	cache and trash bytes freed, in KiB
`errors`	number of managers that returned non-zero
`reboot`	1 if a reboot is required, else 0
`managers`	array of `{name, result}` entries

15.2 Prometheus metrics

`up -metrics FILE` reads the most recent entry of `history.jsonl` and writes a Prometheus textfile-collector snapshot. Exposed gauges:

- `emerger_last_run_timestamp_seconds`
- `emerger_last_run_duration_seconds`
- `emerger_last_run_freed_bytes`
- `emerger_last_run_errors`
- `emerger_reboot_required`
- `emerger_manager_ok{manager="..."}` (one per manager)

The flag does *not* trigger an update run. Invoke it from a `post.d` hook or from the scheduler after `up` has completed:

```
# ~/.config/emerger/hooks/post.d/90-prom.sh
up --metrics /var/lib/node_exporter/textfile_collector/emerger.prom
```

15.3 Reboot exit code

By default `up` exits 0 on success even when a reboot is pending (the summary prints `REBOOT RECOMMENDED`). Pass `-reboot-exit` to turn that into exit code **4** instead, so an orchestrator can react:

```
up -y --reboot-exit
case $? in
    0) ;;                                   # done, no reboot needed
    3) notify-send "eMerger: some managers failed" ;;
    4) systemctl reboot ;;                  # clean, reboot required
esac
```

15.4 Download-only and offline

`-download-only` (alias `-offline`) refreshes the package indexes and *downloads* the pending upgrade set but does not install it. Supported natively on:

Manager	Command
pacman	`pacman -Syuw`
apt	`apt-get upgrade -download-only`
dnf	`dnf upgrade -downloadonly`
zypper	`zypper update -download-only`

Other managers ignore the flag.

15.5 Manager filtering

- `-only apt,flatpak` restricts the run to the listed managers (comma-separated).
- `-except snap,fwupd` runs everything that would normally run, minus the listed managers.

Filtering is applied after detection and after `-dev` / `-firmware` gating, so:

- `-only X` with `X` not detected is a no-op (nothing runs).
- `-except` always wins over `-only` when both mention the same name.

Compose with profiles (`up -profile work -only apt`) to restrict a profile on the fly.

16 Auto-update (unattended)

```
up -au              # Linux / macOS
```

```
up -au              # Windows
```

Backends chosen automatically:

- **Linux**: systemd user timer (preferred) at `~/.config/systemd/user/emerger.{service,timer}`; cron fallback when `systemctl -user` is not available.
- **macOS**: cron fallback via `crontab -l` (you can use `launchd` manually if you prefer).
- **Windows**: `Register-ScheduledTask -TaskName eMerger`.

The trigger fires weekly on Sunday at 10:00 (local time) with ±1h randomised delay, and always uses `-y -q -nl -ni`.

16.1 Managing the schedule

```
# Linux
systemctl --user status emerger.timer
systemctl --user stop emerger.timer
systemctl --user disable emerger.timer
```

```
# Windows
Get-ScheduledTask eMerger
Unregister-ScheduledTask eMerger
```

To avoid night-time runs, pair with `QUIET_HOURS` in `config.sh`. Scheduled runs started inside the window exit immediately; interactive ones are unaffected.

17 Cookbook

Concrete, copy-pasteable recipes for common situations.

17.1 Daily driver (Linux desktop)

```
up -v
```

Simple. Live output, everything updated, cache and trash wiped at the end. Add `-firmware` occasionally.

17.2 Unattended server

Install the schedule once:

```
cat > ~/.config/emerger/config.sh <<'EOF'
ARG_SECURITY=1
ARG_YES=1
ARG_NO_CACHE=1
ARG_NO_TRASH=1
QUIET_HOURS="08:00-18:00"
EOF
up -au
```

You now get weekly security-only updates outside working hours. Pair with Prometheus:

```
# post.d hook
up --metrics /var/lib/node_exporter/textfile_collector/emerger.prom
```

17.3 Developer workstation

Everything, every time - including dev toolchains and firmware:

```
up --dev --firmware --parallel -v
```

Or as a profile:

```
up --profile home
```

17.4 Pre-demo machine

The day before a presentation you want security patches but no big user downloads and definitely no reboots:

```
up --profile safe
```

17.5 Metered / mobile connection

Prefetch when you're on Wi-Fi:

```
up --download-only -y
```

Then, later, without network:

```
up -y
```

Native package managers will use the local cache populated by the earlier run.

17.6 CI runner / container image builder

```
up --only apt -y -qq --reboot-exit
rc=$?
if (( rc == 4 )); then echo "reboot required"; exit 0; fi
exit $rc
```

17.7 Gated maintenance window

Snapshot, update, rollback on failure, reboot on success:

```
set -e
up --snapshot -y || { up --rollback; exit 1; }
up -y --reboot-exit
if [[ $? -eq 4 ]]; then systemctl reboot; fi
```

17.8 Slack-notified runs

```
# ~/.config/emerger/hooks/post.d/99-slack.sh
#!/usr/bin/env bash
state=~/.local/state/emerger
payload=$(tail -1 "$state/history.jsonl")
curl -sS -X POST -H 'Content-Type: application/json' \
    -d "{\"text\":\"eMerger: $payload\"}" "$SLACK_WEBHOOK_URL"
```

17.9 macOS with Homebrew only

```
up --only brew,mas -v
```

17.10 Windows without Chocolatey

```
up --except choco -v
```

17.11 Audit a single package

Is a specific package about to change? Preview the plan and read its upstream changelog:

```
up -n | grep -i firefox
up --changelog firefox
```

17.12 Pinning a package (Linux)

Add to `~/.config/emerger/ignore.list`:

```
# ~/.config/emerger/ignore.list
nvidia-driver-535
```

Then, for `apt`:

```
sudo apt-mark hold nvidia-driver-535
```

(pacman respects the file natively.)

18 Security model

- eMerger is a client-side tool. It has no network listener, no daemon, no persistent background process.
- It reads and writes only three locations: its config dir (`~/.config/emerger/`), its cache dir (`~/.cache/emerger/`) and its state dir (`~/.local/state/emerger/`), plus the global lock at `/tmp/emerger.lock` on Unix.
- `sudo` credentials are cached by `sudo` itself, not by eMerger. When the run ends, the cached credentials expire on the usual `sudo` schedule (5 minutes by default).
- On Windows, UAC elevation happens *only* when a detected manager actually needs admin. The elevation is logged.
- Hooks and plugins run as the invoking user. They can do anything the user can do - pin them to inodes you trust.
- The only network calls eMerger itself makes are (a) the optional weather widget via `wttr.in`, (b) `git pull` during `-self-update`, (c) `fwupdmgr refresh` when `-firmware` is active. Everything else is delegated to the package manager you asked for.
- There is no telemetry. `history.jsonl` stays on your machine unless a hook you wrote ships it elsewhere.

19 Files and paths

Linux and macOS

Path	Purpose
`~/.config/emerger/config.sh`	User defaults
`~/.config/emerger/profiles.d/`	User profiles
`~/.config/emerger/hooks/{pre,post}.d/`	Hook scripts
`~/.config/emerger/ignore.list`	Ignore list (pacman native)
`~/.config/emerger/managers.d/*.sh`	User manager plugins
`~/.cache/emerger/detected`	Detection cache
`~/.local/state/emerger/emerger.log`	Log (rotated at 2000 lines)
`~/.local/state/emerger/history.jsonl`	One JSON record per run
`~/.local/state/emerger/pkgs.before`	Package list snapshot (pre)
`~/.local/state/emerger/pkgs.after`	Package list snapshot (post)
`~/.local/state/emerger/pkgs.diff`	Last-run package diff
`~/.local/state/emerger/resume`	Resume cursor
`/tmp/emerger.lock`	Global lock (flock)

Windows

Path	Purpose
`%APPDATA%\emerger\config.ps1`	User defaults
`%APPDATA%\emerger\profiles.d\`	User profiles
`%APPDATA%\emerger\hooks\{pre,post}.d\`	Hook scripts
`%LOCALAPPDATA%\emerger\cache\`	Detection cache
`%LOCALAPPDATA%\emerger\state\emerger.log`	Log
`%LOCALAPPDATA%\emerger\state\history.jsonl`	Run history

20 Supported package managers

- **Linux, system (need sudo):** `pacman`, `apt`, `apt-get`, `dnf`, `yum`, `zypper`, `xbps`, `apk`, `eopkg`, `emerge`, `nixos-rebuild`, `fwupdmgr`, `snap`.
- **Linux, AUR (no sudo):** `yay`, `paru`.
- **Linux, user-space:** `flatpak`, `nix-env`.
- **macOS:** `softwareupdate` (native), `brew`, `brew -cask`, `mas`.
- **Windows:** `winget`, `scoop`, `choco`, `PSWindowsUpdate`, `wsl -update`.
- **Dev toolchains** (all platforms, opt-in with `-dev`): `rustup`, `cargo install-update`, `npm`, `pnpm`, `pip`, `gem`.

21 Exit codes

Code	Meaning
0	Success
1	Runtime failure (sudo, lock, disk, interrupted)
2	Argument parsing error
3	One or more package managers returned non-zero
4	Reboot required (only with `-reboot-exit`)

Useful idioms:

```
up -y -q || case $? in
    3) notify-send "eMerger: some managers failed" ;;
    *) logger -t eMerger "fatal $?" ;;
esac
```

```
up -y -q
if ($LASTEXITCODE -eq 3) { Write-Warning "Some managers failed" }
```

22 Troubleshooting

"Another eMerger run is in progress". A stale `flock` on `/tmp/emerger.lock`. Inspect `ps` for stragglers, then remove the lock file.

A manager fails with exit code 3. Run `up -errors` to tail the log, then `up -doctor` to inspect the environment. Most failures come from a manager's own health state (for example `dpkg` interrupted); run the native repair command (`sudo dpkg -configure -a`, `sudo pacman -Dk`) before retrying.

Nothing happens on scheduled runs. Check `systemctl -user status emerger.timer` or the Windows Task Scheduler entry. On Unix the scheduled run always uses `-y -q -nl -ni`; enable `-v` in the unit for one cycle to see the output. On Linux systemd user timers do not fire if no session is open unless `loginctl enable-linger` has been run for the user.

Emoji renders as boxes or question marks. The terminal font does not include Unicode ranges eMerger uses. Either pass `-no-emoji`, set `ARG_NO_EMOJI=1` in `config.sh`, or switch to a Unicode-capable font.

Spinner disappears on terminal resize. Harmless. The live-log width recomputes every 120 ms.

`up -self-update` aborts with "non fast-forward". You have local commits on top of `main`. Rebase or reset them manually; eMerger refuses to clobber local work.

`notify-send` (Linux) never shows a notification. No `DISPLAY` / `WAYLAND_DISPLAY` in the environment (typical for cron). Use a systemd user timer instead: it inherits the session. Alternatively, write a `post.d` hook that notifies your own channel (Slack, Telegram, Discord, email).

(Windows) "up : The term 'up' is not recognized". Your PowerShell profile did not load. Run `. $PROFILE` or open a new window. If still missing, re-run `.\setup.ps1`.

(Windows) "cannot be loaded because running scripts is disabled". `ExecutionPolicy` is `Restricted`. Lift it for the user:

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

(Windows) Elevation fails silently. You canceled the UAC prompt. The script logs `relaunching elevated` and exits: the elevated window does the actual work.

Pacman keeps asking about ignored packages. `ignore.list` is passed as `-ignore=`; `pacman` still prints the warning line. This is upstream behaviour, not an eMerger bug.

"command not found: gum" or "whiptail". The interactive menu (`-i`) falls back to a plain-read loop when neither tool is installed. Install `gum` (https://github.com/charmbracelet/gum) for a nicer UI, or ignore the warning.

23 Frequently asked questions

Is eMerger safe for production servers? Yes, but use it in "security only, unattended" mode: `up --profile server` (or its inlined equivalent). The `-reboot-exit` idiom lets your orchestrator decide when to reboot.

Does `up` install packages I don't already have? No. `up` only invokes managers that are already installed and only asks them to upgrade what is already installed. No new software is added unless a package upgrade pulls a new dependency in.

Does `up` reboot my machine? Only with `-reboot`. Without it, the summary prints a reboot advisory and the run exits normally.

Can I use `up` without `sudo`? Yes, in user-only mode: `up --only flatpak,brew,npm,pip,cargo` (adjust to what you have). User-level managers never need elevation.

Why is my terminal flashing during the parallel run? Under `-parallel` each user-space manager streams its own output; when `-v` is not set, output is buffered and flushed at the end. If you want a calm terminal, drop `-v` or narrow the parallel set with `-only`.

Can I use `up` inside a Docker image build? Yes. Use `up --only apt -y -qq` (or the relevant manager) and ignore the reboot advisory. Avoid `-firmware`, `-snapshot` and interactive flags.

What happens if two `up` runs start at the same time? The second one aborts with exit code 1 ("Another eMerger run is in progress"). The lock is a plain `flock` on `/tmp/emerger.lock`.

How do I completely remove eMerger? Run the uninstaller, then:

```
rm -rf ~/.config/emerger ~/.cache/emerger ~/.local/state/emerger
rm -rf /path/to/eMerger     # the repo itself
```

Where is my data? All local, all in the paths listed in section 19. Nothing leaves your machine unless a hook you wrote ships it.

Can I contribute a new package manager? Absolutely - see section 25. Most contributions are a 20-line dispatch branch plus a test case.

24 Glossary

- **Dry-run** - a simulation: the tool prints the commands it would run without actually running them.
- **Hook** - a user script that runs before (`pre.d`) or after (`post.d`) the update flow.
- **Manager (package manager)** - software such as `apt`, `pacman`, `brew`, `winget` that installs, upgrades and removes programs on your machine.

- **Parallel mode** - concurrent execution of user-space managers. Safer than it sounds, because user-space managers do not share a lock.
- **Plugin** - a user-provided manager definition. Registers a new manager without modifying the repository.
- **Profile** - a named bundle of default flags, loaded via `-profile NAME`.
- **Resume cursor** - a file that records which managers completed successfully, consumed by `-resume`.
- **Snapshot** - a read-only filesystem checkpoint taken before the upgrade. Linux only (snapper/-timeshift/btrfs).
- **TUI** - Terminal User Interface. The interactive menu (`-i`) on Unix.
- **UAC** - User Account Control, the Windows elevation prompt. eMerger relaunches itself elevated when needed.

25 Development

25.1 Repository layout

```
eMerger/
  src/
    emerger.sh              # Unix entry (Linux + macOS)
    emerger.ps1             # Windows entry
    lib/                    # bash libs
    pslib/                  # PowerShell libs
    logo/
  share/profiles/           # shipped profiles (*.sh + *.ps1)
  share/plugins/            # example plugins
  completions/              # bash/zsh/fish completions
  tests/                    # bats tests
  man/up.1                  # man page
  setup.sh  setup.ps1
  uninstall.sh  uninstall.ps1
  update.sh  update.ps1
  VERSION
  .github/workflows/ci.yml
```

25.2 Bash library modules (`src/lib/`)

File	Role
`ui.sh`	Colors, glyphs, spinner, box, live-log monitor
`log.sh`	Structured logging (rotated at 2000 lines)
`sys.sh`	OS / shell / battery / disk detection
`run.sh`	Command runner (dry-run, retry, progress)
`args.sh`	Argument parser and short-bundle expander
`packages.sh`	Per-manager dispatcher
`clean.sh`	Cache and trash cleaners
`hooks.sh`	User hook runner
`update.sh`	Self-update + cron / timer setup
`notify.sh`	Desktop notifications
`summary.sh`	Final banner, history persistence

File	Role
`tui.sh`	Interactive menu
`lock.sh`	Global `flock`
`retry.sh`	Retry on transient failures
`reboot.sh`	Reboot-required detection
`diff.sh`	Package snapshots + diff
`disk.sh`	Disk-space precheck
`snapshot.sh`	snapper / timeshift / btrfs
`mirrors.sh`	Mirror rank / refresh
`resume.sh`	Resume cursor
`doctor.sh`	`-doctor`
`changelog.sh`	`-changelog PKG`
`report.sh`	Markdown export
`wizard.sh`	First-run wizard
`profiles.sh`	Profile loader
`progress.sh`	Output summary + highlight
`estimate.sh`	Step ETA from history
`ignore.sh`	Ignore list loader
`plugins.sh`	User plugin loader
`metrics.sh`	Prometheus textfile export
`help.txt`	`up -help` content

25.3 PowerShell library modules (`src/pslib/`)

File	Role
`UI.ps1`	Colors, glyphs, box, step
`Log.ps1`	Structured logging
`Sys.ps1`	OS, admin, battery, disk, UAC elevation
`Args.ps1`	Arg parser (shift + regex)
`Packages.ps1`	Manager dispatcher + `Run-Cmd`
`Clean.ps1`	`%TEMP%` and Recycle Bin
`Hooks.ps1`	`hooks\{pre,post}.d\*.ps1`
`Update.ps1`	`git pull` self-update + Task Scheduler
`Notify.ps1`	BurntToast (optional)
`Summary.ps1`	Final box, history, reboot detection
`Doctor.ps1`	`-doctor`
`Profiles.ps1`	Profile loader
`Help.txt`	Help text

25.4 Running tests

```
sudo apt-get install bats shellcheck
bats tests/
shellcheck -S error src/emerger.sh src/lib/*.sh setup.sh uninstall.sh update.sh
```

Both run on push via `.github/workflows/ci.yml`.

25.5 Adding a package manager

Unix - edit `src/lib/packages.sh`:

1. Add the name to `PKG_MANAGERS`.
2. Add a branch in `_pkg_detect_raw`.
3. Add a branch in `pkg_run` with `run_cmd` calls.
4. If the manager does not need sudo, exclude it from `pkg_needs_sudo`.
5. Optional: add an emoji in `pkg_icon`, a regex in `progress.sh` to parse its output, a completion entry.

Windows - edit `src/pslib/Packages.ps1`:

1. Add the name to `$script:PKG_MANAGERS` (or `PKG_DEV`).
2. Add a `Pkg-Detect` case.
3. Add a `Pkg-Run` case with `Run-Cmd` calls.
4. If it needs admin, add it to `Pkg-Need-Admin`.

Both files are table-driven dispatchers; most contributions end up being a twenty-line patch plus a test case under `tests/`.

25.6 Contributing

Pull requests and issues are welcome on GitHub. Before opening a PR:

- Run the test suite in `tests/` on the platform you touched.
- Attach `up -doctor` output and the relevant chunk of the log when reporting bugs.
- Keep feature parity in mind: when a feature makes sense on both Bash and PowerShell, please implement it on both sides or flag it for follow-up.

If your question is not answered here, open an issue on GitHub: we would rather improve the documentation than answer the same question twice.